



Voice-Controlled Robot

by

Joshua O'Brien

This report is submitted in partial fulfilment of the requirements of the Degree in Electronic and Communications Engineering (TU714) of the Technological University Dublin

April 13th, 2026

Supervisor: Dr. Nouman Ashraf

School of Electrical and Electronic Engineering

DECLARATION

I, the undersigned, declare that this report is entirely my own written work, except where otherwise accredited, and that it has not been submitted for a degree or other award to any other university or institution.

Signed: Joshua O'Brien

Date: 02/03/2026

Table of Contents

Table of figures	3
1.0 Introduction.....	5
1.1Voice recognition	5
1.1 Applications of voice recognition	6
1.1.1 Assistive living technologies	6
1.1.2 Smart home and IOT devices	6
1.2 Project outline	6
1.3 Machine learning and neural networks	6
2.0 Development	8
2.1 System design	8
2.2 Robotics design.....	9
2.2.1 Processor	9
2.2.2 Motors and motor drivers	9
2.2.3 Distance sensor	10
2.2.4 Battery management.....	11
2.3 Software design	12
2.3.1Voice recognition architecture.....	12
2.3.2 Programming language	13
2.3.3 Audio processing	13
3.0 Implementation	13
3.1 Implementation of audio processing	13
3.2 Implementation of Convolutional Neural Network	16
3.2.1 Dataset preparation	17
3.2.2 CNN architecture.....	18
3.3 Real time audio processing.....	20
3.3.1 Flutter application	20
3.3.2 Python webserver	21
3.4 Hardware implementation	21
3.5 Implementing robot movement	23
4.0 Testing	25
4.1 ESP-32 with motor driver and motor	25
4.1.1 Motor test 1	25
.....	25
4.1.2 Motor test 2	26
4.1.3 Motor test 3	26

4.2 ESP-32 with HC-SR04	27
4.3 Stopping test.....	27
4.4 Bluetooth connectivity	28
4.5 CNN testing.....	29
4.6 Integration testing.....	30
4.6.1 System test.....	30
5.0 Conclusion	31
5.1 Evaluation	31
5.2 Future improvements	31
5.2.1 Edge computing	31
List of acronyms	31
References.....	32
Appendices.....	35
Appendix 1: Source code	35
Appendix 2: Log book	36

Table of figures

Figure 1 process of speech recognition from spoken word to the predicted output	5
Figure 2 a model of an artificial neuron [10].....	7
Figure 3 layers present in a neural network [12].....	8
Figure 4 System design diagram.....	9
Figure 5 ESP32 S3.	9
Figure 6 TB6612FNG.	10
Figure 7 HC-SR04 ultrasonic distance sensor.	11
Figure 8 6V battery pack.	11
Figure 9 3.7V lipo battery.....	12
Figure 10 lipo rider plus.	12
Figure 11 preemphasis difference filter implemented in python.....	13
Figure 12 Python code to calculate the number of frames needed to make up a 1 second word.	14
Figure 13 python function to perform zero padding.	14
Figure 14 rearranging of frames into an array and application of a hamming window on the audio frames.....	14
Figure 15 code to compute the magnitude and then power spectrum of the FFT in python.	14
Figure 16 mel filterbank plotted in the frequency domain.....	15
Figure 17 create the points where each filter starts and ends in the frequency domain.....	16
Figure 18 construction of triangular filters in python.....	16
Figure 19 final step in computing the log mel spectrogram pf a one second audio file	16
Figure 20 data preparation for CNN.	17
Figure 21 splitting of test and validation data.....	17

Figure 22 compute mel spectrogram prior to CNN training.	18
Figure 23 normalisation prior to training.....	18
Figure 24 Convolutional layers of the CNN.	19
Figure 25 final layers of the CNN.	19
Figure 26 CNN callback function.	20
Figure 27 Model of CNN.	20
Figure 28 UI of mobile application.	21
Figure 29 MDF board with motors, wheels and ball casters attached.	22
Figure 30 schematic view of TB6612FNG (U2), n20 motor left (J2), n20 motor right (J1) and ultrasonic distance sensor (U3).	22
Figure 31 schematic view of esp32 (U1).	23
Figure 32 schematic view of lipo rider plus (J5), 3.7V lipo battery (J3) and 6V battery pack (J4).	23
Figure 33 Robotic unit.....	23
Figure 34 service UUID and RX characteristic UUID for Nordic UART.....	24
Figure 35 MyCallbacks class Arduino.....	24
Figure 36 BLE setup in Arduino.	25
Figure 37 Output in serial monitor of motor rotation test.	25
Figure 38 Output from motor test 2.	26
Figure 39 Output for motor test 3.	26
Figure 40 Output from HC-SRO4 distance test.....	27
Figure 41 Output from motor stopping test.	28
Figure 42 ESP32 advertising its BLE connection.	28
Figure 43 Output from BLE testing.	29
Figure 44 Confusion matrix output at the end of CNN training.	29
Figure 45 Confusion matrix tested using data outside of Google speech dataset.	30
Figure 46 Noisy word prediction confusion matrix.	30

1.0 Introduction

1.1 Voice recognition

The implementation of voice recognition is core to achieving a successful project outcome. Speech to text (STT) and particularly automatic speech recognition (ASR) which is the ability of a machine to convert a speech input to text in real time [1]. As this robot only requires a certain number of keyword commands continuous speech recognition won't be necessary with isolate word recognition most suited.

These systems begin by taking a continuous analog audio signal and converting it to a digital signal. The analog to digital conversion carries out two processes, sampling and quantisation. Sampling being taking a continuous time signal and breaking this into individual samples of the signal with the rate at which these samples are taken being at least twice the maximum frequency desired to be captured [2]. Quantisation is used to convert the magnitude into the digital domain by mapping it to discrete levels e.g. 8 bit quantisation gives 2^8 levels or 256 levels.

Once the audio has been digitised feature extraction is performed to extract the meaningful data used to identify the words. Once feature extraction has been performed this data can then be used to predict the spoken word using through algorithm. Initially ASR used dynamic time warping algorithms (DTW) to predict spoken word though pattern matching to stored templates [3]. These ASR algorithms are often assessed using word error rate (WER) which is the rate at which an error occurs as a percentage of the total words predicted. An example of a modern model which achieved a low WER is whisper by OpenAI with a word error rate of 2.5% under ideal circumstances, this model however required massive amounts of training time as there was approximately 680,000 hours of audio used in training [4]. For low resource isolated keyword recognition models such as convolutional neural networks (CNN) or Hidden Markov models (HMM), are much better suited. Figure 1 shows a block diagram of the process to take a continuous speech signal and perform ASR on it.

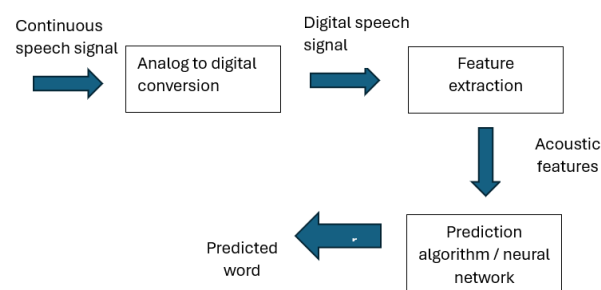


Figure 1 process of speech recognition from spoken word to the predicted output.

1.1 Applications of voice recognition

Isolated speech recognition and its use in robotics have with applications with fields such as assistive living technologies, automate call services and smart home devices.

1.1.1 Assistive living technologies

For users with severe mobility impairments operating a wheelchair can be extremely difficult or impossible. With the addition of voice commands to a motorised wheelchair this could give users the ability to operate hands free and give greater independence to those with physical impairments. By integrating an ASR system with directional keywords such as “forward”, “backward”, “left and “right” the user could gain a heightened level of autonomy in comparison to a traditional motorised wheelchair operated via joystick. Low latency as well as object detection are critical in ensuring safety in this implementation.

1.1.2 Smart home and IOT devices

In the past decade smart home devices have become extremely popular. Devices such as Amazon Alexa and Google home use ASR to allows user to control lights, temperature or music in their home through their voice only. This trend of growth of these smart home devices with speech recognition built in has increased the implementation of voice recognition and speech to text in smart devices over the last decade.

1.2 Project outline

This project consists of two main components these being the voice recognition module along with the robotics unit. These components should be integrated seamlessly so that the voice commands are processed by the speech recognition module and then transmitted in real time to the robotics unit where the commands will then be realised. There are 11 words which have been chosen as commands for this project

“left”, “right”, “backward”, “forward”, “start”, “stop”, “one”, “two”, “three”, “four” and “five”.

The objectives of this project are to achieve a high level of accuracy with identifying these words. There latency between the word being spoken and action . The robot should have an accurate object avoidance mechanism to avoid collisions. The robot should have a high level of manoeuvrability capable of performing turns and reversing efficiently. The robot should have the capability to run for an extended period on one charge.

1.3 Machine learning and neural networks

Machine learning (ML) is the term used to describe the ability of computers to learn without explicit programming [5]. This is a branch of artificial intelligence (AI) which is used across many fields such as speech recognition, image classification and natural language processing. Machine learning can be further broken down into 3 categories Supervised learning, unsupervised learning and reinforcement learning.

Supervised learning is used when the data has already had target values assigned to it [6]. This is the type of learning which was implemented in this project where data had already been labelled with its corresponding value e.g. the audio files for the word 2 labelled as 2. Unsupervised learning is performed to determine the hidden patterns or groupings in the data[6]. Reinforcement learning

then uses trial and error where positive and negative feedback is given based on the outcome of an action e.g. a robot learning where objects are in a room through bumping into them [7]. Supervised learning was used in the implementation of this project. This was possible due to the dataset mentioned in section 1 is labelled and prepared by google based off the command word.

Neural Networks (NN) are networks modelled after the behaviour of the human brain. These algorithms work by creating artificial neurons intended to mimic the neurons present in a human brain shown in figure 1. These neurons are then interconnected and arranged in layers shown in figure 2. The neuron initially takes inputs which are then multiplied by a learned weight, these weighted inputs are then summed along with a bias which then passes through a activation or transfer function to give the output as is shown in figure 1. The formula below describes the mathematical function of an artificial neuron.

$$y = \varphi(\sum_{i=1}^n w_i x_i + b)$$

Where y = output, n = number of inputs, φ = activation function, w = weight, x = input and b = bias [8].

The purpose of the weights is to influence how each input contributes to the output of the neuron, a high weight will mean the input has significant effect on the output and a low means it contributes little. The bias then acts as a constant a high bias value will then trigger the neuron activation with low values from the other weighted input while a low bias will cause the neuron to require much higher weighted inputs to be activated [9]. The activation function then applies a function to sum to determine the final output, this determines whether the neuron fires and to what degree.

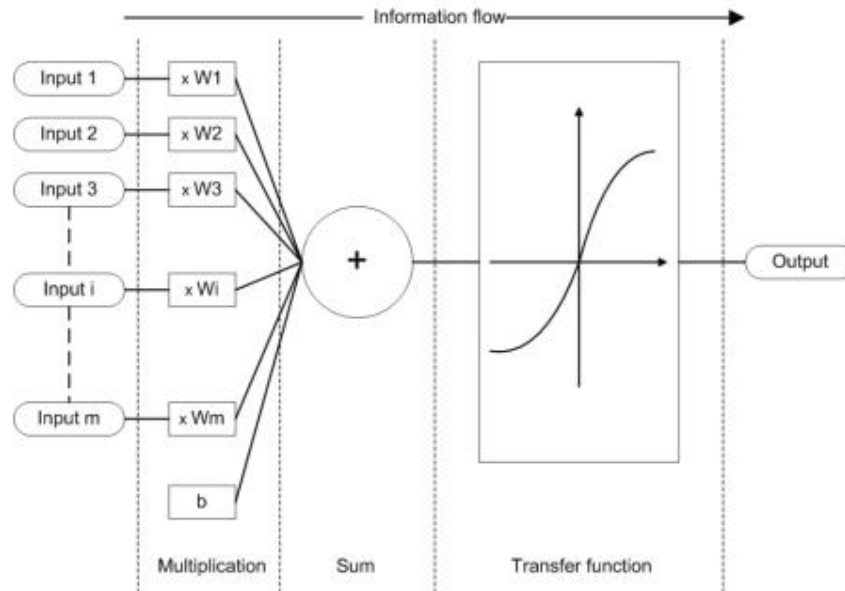


Figure 2 a model of an artificial neuron [10].

To create a neural network, neurons are connected in several layers. The layers of a neural network are broken down into 3 types of layers, the input layer receives the raw data, the hidden layers then process the data which is then passed to the output layer to give the final prediction shown in figure 2. Each neuron in a previous layer is then connected to each neuron in the next. When more than one hidden layer is implemented, this is considered a deep neural network [11].

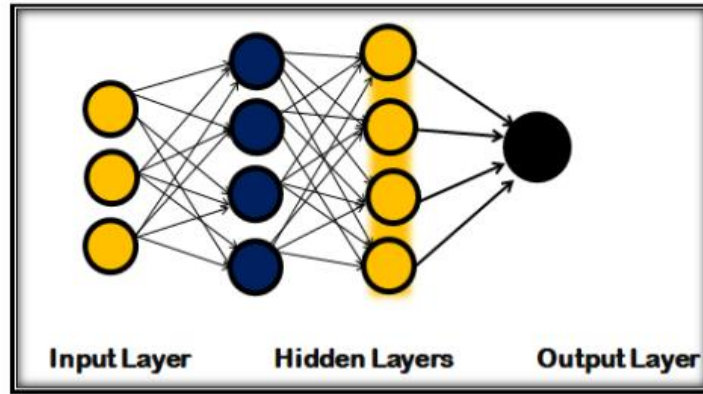


Figure 3 layers present in a neural network [12].

Training is the process of repeatedly adjusting the weights of a neural network to minimise the error between the predicted and actual output. Training is achieved through the process of forward propagation and backpropagation. Forward propagation is the regular operation of a neural network where data is passed to the input layer and then processed through all layers to arrive at a final prediction. The loss function is then used to calculate the error between the predicted value and true value [13]. Backpropagation is then employed which works through the network calculating the gradient of loss with respect to each weight [12]. The gradient is the rate of change of loss with respect to the weight mathematically defined as.

$$\frac{dL}{dw}$$

Where L = loss and w = weight.

An optimiser function is then applied to update the weights based off the calculated gradient values. The gradient descent optimiser is a simple function which takes the gradient and updates the weights in the opposite direction of the gradient. Learning rate is the factor by which the weights will change in relation to the gradient.

Overfitting is an important consideration when training a neural network as it can lead to inaccuracy of a model to new data inputs while maintaining high accuracy with training data [14]. This is caused by the model learning patterns such as noise or random variations instead of generalised features of the data. Methods such as early stopping, dropout and regularisation can be used to minimise overfitting these will be discussed further in section 3.2.

2.0 Development

2.1 System design

The initial system design seen in figure 4 proposed a system consisting of 3 distinct components. The components included were a Python server which acted as the centre of the speech processing process taking in the unprocessed data of a vocal recording and outputting a command word representing the identified word. The mobile application acted as the main interface between user and system taking their voice commands as recording to be further processed. The robotics unit then would take the processed command to control the speed and direction of the robot.

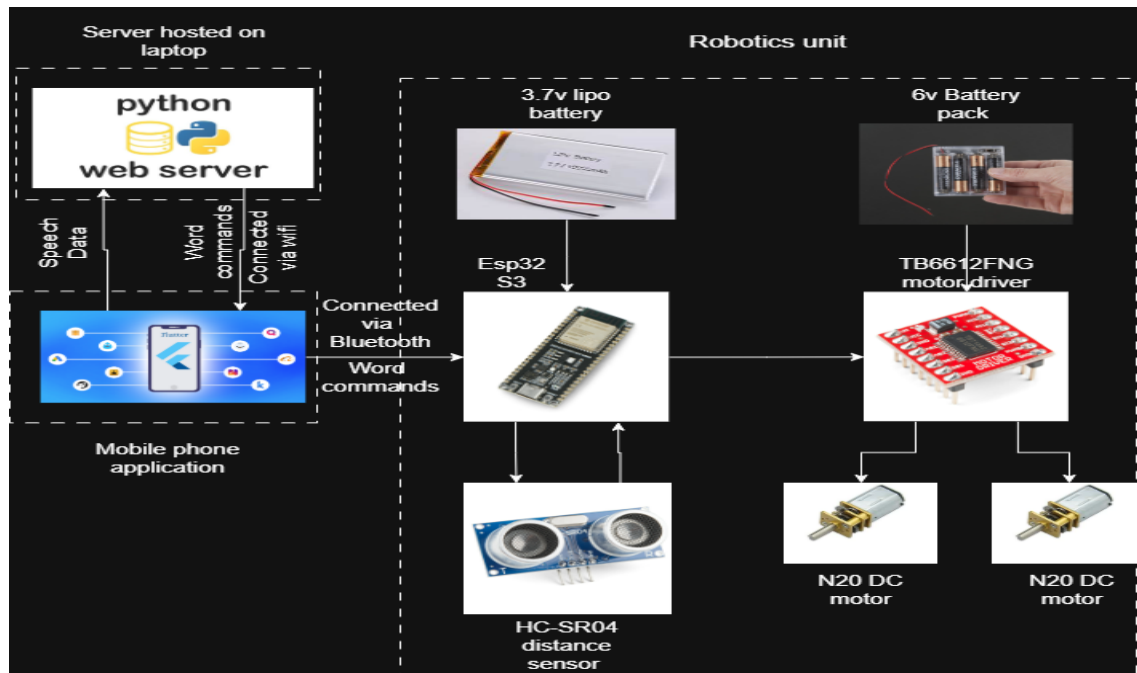


Figure 4 System design diagram.

2.2 Robotics design

2.2.1 Processor

The chosen microcontroller for this project was the esp-32-S3. Connectivity, memory and expandability were the main criteria for which this choice was made. The connectivity of the esp was a driving factor in its choice with WI-FI connectivity and Bluetooth Low Energy (BLE) this microcontroller had the ability to connect to the application via Bluetooth or wi-fi [15]. There is also roughly 8MB of space available for storing the program on external flash memory. There is also 40 programmable GPIO pins on this board giving the option to add sensors and modules as the project progressed. Based off the necessity for connectivity for this project other microcontrollers such as the Arduino Uno R3 require separate wifi and Bluetooth modules with only 20 GPIOs they were quickly discounted [16].



Figure 5 ESP32 S3.

2.2.2 Motors and motor drivers

The chosen motor driver for this project was the TB6612FNG. This is a H-Bridge motor driver with the ability to drive two DC motors simultaneously. This motor driver works on a logic level of 3.3V

ensuring it works directly with the esp-32 [17]. This motor driver works off MOSFET transistors in comparison to the BJTs in an L298N driver [18]. The MOSFETS require a much lower voltage drop roughly 0.5V vs 3V in the L298N this leads to a higher efficiency and more voltage reaching the motors. The TB6612FNG also requires no heat sink due to low power dissipation leading to a much smaller form factor.

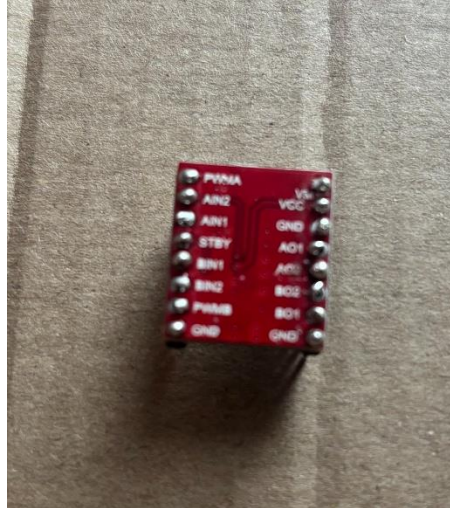


Figure 6 TB6612FNG.

The wheel motors chosen to be paired with the motor driver were N20 geared DC motors. These motors work with a supply voltage in the range of 3V and 12V [19]. They were chosen due to the precision of the motors as well as the feedback from the magnetic encoders attached to these motors.

2.2.3 Distance sensor

To give this robot the ability to sense its surroundings and prevent collisions it was necessary to add a distance sensor. The distance sensor chosen was the HC-SR04 which is an ultrasonic distance sensor [20]. This sensor consists of a transmitter T and receiver R the transmitter when triggered via the trigger pin will then transmit a burst of 8 ultrasonic pulses at 40kHz. This signal will reflect when it hits an object and reflect back to the receiver. The receiver then sends a high pulse via echo pin to the microcontroller for the duration the signal took to get from transmitter to receiver. Based off this information it is possible to calculate distance through the formula below.

$$D = \frac{T * 0.0343}{2}$$

Where D = distance, T = time in μs .

This sensor has a measurement range of between 2cm and 400cm making it ideal for detecting objects in the short to medium range. The main issue with this sensor is its operating voltage of 5V where the microcontroller operates at 3.3V. There is an iteration of this sensor made with 3.3V however this model was unavailable. The trigger pin on this sensor should respond to the 3.3V input from the esp-32 however issues would arise when the echo pin sends 5V straight to the esp-32s GPIO pin likely causing damage to the board as its datasheet doesn't recommend an input voltage above 3.6V or VDD +0.3V. This was solved by adding a simple voltage divider of a 1K Ω and 2K Ω resistors.

General formula:

$$V_{out} = V_{in} * \frac{R2}{R1 + R2}$$

Actual implementation:

$$3.33 \dots V = 5V * \frac{2k}{1k + 2k}$$

This will solve the issue of the mismatched logic levels allowing this sensor to be used.



Figure 7 HC-SR04 ultrasonic distance sensor.

2.2.4 Battery management

Two batteries were chosen to for use in this project a 6V battery pack of 4 AA batteries was chosen for powering the motors via the motor driver. A separate 3.7V LIPO battery was used to power the esp-32 via a seed studio Lipo rider plus module [21] . The power supplies for motors and microcontroller were separated for a number of reasons. The use of separate batteries reduces the electrical noise generated by the DC motors from interfering with the esp32. The large current draw created by startup and or change in motor direction could cause voltage dips to the esp32 if one power supply is used. A voltage drop below the operating voltage of the esp32 would result in complete loss of control of the robot and the Bluetooth disconnecting.

Given the esp32 operates at a 5V logic level a boost converter was needed to boost the voltage from the 3.7V lipo battery. The seed studio lipo rider plus supports an output of both 5V and 3.3V which can then be output to the esp32. This device combines both charging as well as regulating voltage making the esp-32 now compatible with the 3.7V lipo battery.



Figure 8 6V battery pack.



Figure 9 3.7V lipo battery.



Figure 10 lipo rider plus.

2.3 Software design

2.3.1 Voice recognition architecture

Modern improvements to continuous speech recognition such as whisper by OpenAI[1], which achieved WER as low as 2.5%, require massive amounts of resources and training time with OpenAI's model using 680,000 hours of training data. For this project a smaller model is required as the dataset is quite small, as previously illustrated in section 1.1 this project matches the criteria for isolated speech recognition in comparison to continuous speech recognition. APIs such as VOSK [22] or previously mentioned whisper were considered however a simpler classification architecture was chosen.

Traditional methods of isolated keyword spotting was implemented using DTW previously mentioned in section 1.1. DTW algorithms however perform poorly across different noise levels and accents due to the bias introduced by using a comparison between two speech signals to classify the spoken word [3]. A CNN based architecture was chosen. The CNN architecture provides a lightweight framework providing accurate performance with short training times. A recent paper on keyword spotting on edge devices [23] found a 94.48% accuracy on clean data using a CNN trained on google V0.02 speech command dataset. Given this speech dataset was chosen for this project along with low computational demands of CNNs this approach was chosen.

2.3.2 Programming language

Once the method of machine learning had been chosen, software was then chosen to implement it. Python was chosen as the language to use for implementing the speech recognition for several reasons. The main reason python was chosen was based off the accessibility to libraries such as Keras and TensorFlow for building machine learning algorithms and NumPy for computation. These libraries have prebuilt functions such as the Keras sequential model [24] for implementing several types of NN such as CNNs. Python also has a simple readable syntax which permits easily changeable and efficient programming during the development process with a faster learning curve in comparison to lower-level languages.

The language chosen for the application was dart through Flutter which is an open source, multi-platform development platform for applications [25] . Flutter was chosen as it provides an environment which is simple to use as well as intuitive. It gives more control over the application in comparison to software such as MIT app inventor.

2.3.3 Audio processing

The audio processing section of this project is key in the implementation of a custom speech recognition software. As this software is using a CNN as the underlying model performing speech recognition the audio data must be converted into a suitable format for the CNN to read. CNNs require an array as an input compared to the current pulse code modulated (PCM) data output from the flutter application. The chosen format of data to be passed to the CNN was a mel spectrogram of each word.

3.0 Implementation

3.1 Implementation of audio processing

As previously discussed in the design and research sections before the audio data could be passed to the CNN for training the data must be processed into an acceptable format. The format chosen as previously mentioned is the mel spectrogram, this was processed inside of a python script written with aid of a guide [26]. This is a process which relied heavily on mathematics this has previously been implemented in similar projects such as a voice controlled robotic arm [27]. Below is a mathematical description along with the implementation of mel spectrogram extraction from PCM audio files in python.

The initial step in processing mel spectrograms is known as pre-emphasis. Pre-emphasis in this context is the act of applying a filter to emphasize high frequency voice components as they tend to be diminished in comparison to low frequency components. The equation is as follows:

$$y[n] = x[n] - ax[n - 1]$$

This equation was then inserted into python giving:

```
pre_emphasis = 0.95
emphasized = np.append(audio_array[0],
                        audio_array[1:] - pre_emphasis
                        * audio_array[:-1])
```

Figure 11 preemphasis difference filter implemented in python.

Once pre-emphasis was implemented the next step in was framing. Framing is the act of taking a piece of audio and then breaking it into smaller chunks or frames. The size of these audio files generally between 20ms and 30ms which is short enough to capture changes between the frames. Below is a python script which performs some calculations to determine the number of frames per second long word. 25 ms was chosen as frame length and 10ms as the stride between frames.

```
frame_length = int(round(0.025 * sampling_rate)) # calculates the number of samples in each frame
frame_stride = int(round(0.010 * sampling_rate)) # calculates the number of samples of each frame stride
signal_length = len(emphasized)
num_frames = 1 + int(np.ceil(
    float(np.abs(signal_length - frame_length)) / frame_stride)) # calculates how many frames are needed for the whole signal
```

Figure 12 Python code to calculate the number of frames needed to make up a 1 second word.

Padding was the next step performed on the data. Padding is fundamental in the implementation of many forms of signal processing. This is the addition of zeros or empty signal to ensure that all frames in this case are of equal size. It is essential for CNNs to receive information of the same size and shape [28].

```
pad_length = num_frames * frame_stride + frame_length # since the fr
padded = np.append(emphasized, np.zeros(pad_length - signal_length))
```

Figure 13 python function to perform zero padding.

The information held in these frames is then inserted into an array of which each row is a separate frame. Once the array is created a hamming window is applied. Hamming windows are used on the frames to prevent spectral leakage, this works by smoothing out the edges which is necessary as discontinuities occur at the edges of frames when they are sliced from the original signal [29].

```
#this section creates an array where each row is a frame of audio
indices = (np.tile(np.arange(0, frame_length), reps: (num_frames, 1)) +
    np.tile(np.arange(0, num_frames * frame_stride,
        frame_stride),
        reps: (frame_length, 1)).T)
frames = padded[indices.astype(np.int32)]
frames *= np.hamming(frame_length) # applies a hamming window to account for spectral leakage
```

Figure 14 rearranging of frames into an array and application of a hamming window on the audio frames.

The next step in the process of extracting the mel spectrogram from the audio stream is applying a Fast Fourier Transform(FFT) of the frame array. The FFT filter converts the current time domain signal into the frequency domain. The frequency domain output will now be a number of complex number spread across a number of frequency bins. The power spectrum of this signal was then calculated as the phase is largely irrelevant in this context. Below is the mathematical formula for the power spectrum of an FFT. Where P = power spectrum, N = number of FFT points and FFT(frame) = frame after FFT is applied.

$$P = \frac{|FFT(frame)|^2}{N}$$

```
# Fast fourier tranform
NFFT = 512 # uses a 512 point fft outputting 257 bins
mag_frames = np.absolute(np.fft.rfft(frames, NFFT)) # gets the magnitude spectrum from the fft
pow_frames = (1.0 / NFFT) * (mag_frames ** 2) # converts magnitude spectrum to power spectrum
```

Figure 15 code to compute the magnitude and then power spectrum of the FFT in python.

The next step performed was application of the mel filterbank onto the current array. The scale i was developed to model the perception of sound of the human ear in comparison to the linear frequency scale while was previously computed. As perception of sound is skewed towards low frequencies and higher frequencies are perceived to be diminished the Mel scale accounts for this. The process and equations used are provided below.

Hz to Mel:

$$m = 2595 \cdot \log_{10} \left(1 + \frac{f}{700} \right)$$

Mel to Hz

$$f = 700 \cdot (10^{\frac{m}{2595}} - 1)$$

Initially the number of filters was declared as 40, a low point in the mel scale was then set to 0. The highest point in the mel scale was set by converting the Nyquist frequency into the mel scale. There are then 42 points spaced evenly between the lowest and highest mel values to create 40 triangular filters. These points are then converted back into the frequency domain where they will now be spread using the Mel scale. As is shown in figure 6 the triangular filters are grouped much tighter at lower frequencies leaving them largely un-attenuated while the higher frequencies are attenuated to a larger degree due to a wider spacing of filters.

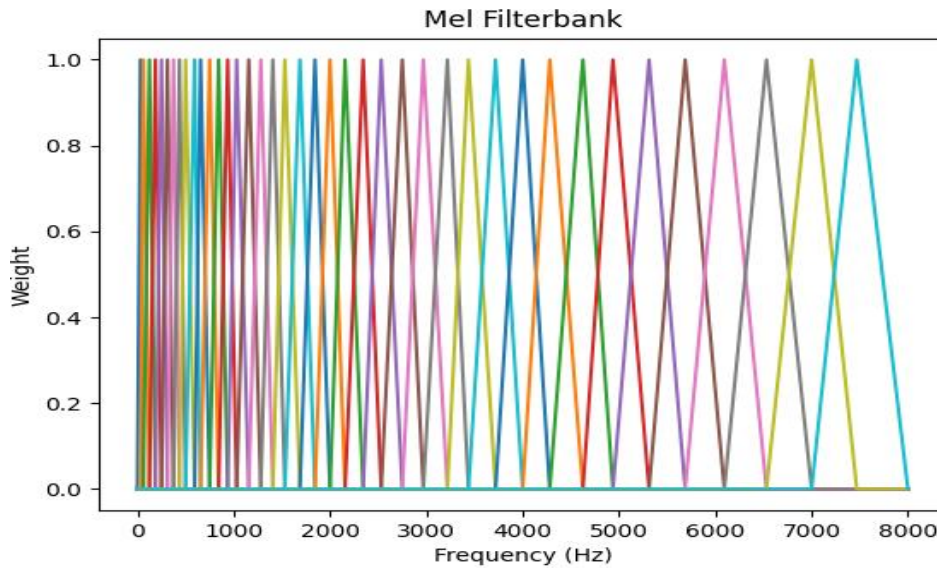


Figure 16 mel filterbank plotted in the frequency domain.

```

# This section converts the power spectrum to mel spectrogram
nfilt = 40
low_mel = 0
high_mel = 2595 * np.log10(1 + (sampling_rate / 2) / 700)
mel_points = np.linspace(low_mel, high_mel, nfilt + 2)
hz_points = 700 * (10 ** (mel_points / 2595) - 1)
bins = np.floor((NFFT + 1) * hz_points / sampling_rate)

```

Figure 17 create the points where each filter starts and ends in the frequency domain.

Once the points needed to build each filter were created the triangular filters were then built. The triangles were then created between the frequency bin points established in figure 18. Each triangular filter starts at zero at point $b(m-1)$, rises to a weight of 1 at $b(m)$ and then descends back to 0 at point $b(m+1)$. Adjacent filters overlap so that the centre of one filter aligns with the boundary of another filter, ensuring full coverage of the frequency spectrum.

```

fbank = np.zeros((nfilt, int(np.floor(NFFT / 2 + 1))))
for m in range(1, nfilt + 1):
    f_m_minus = int(bins[m - 1])
    f_m = int(bins[m])
    f_m_plus = int(bins[m + 1])
    for k in range(f_m_minus, f_m):
        fbank[m-1, k] = (k - bins[m-1]) / (bins[m] - bins[m-1])
    for k in range(f_m, f_m_plus):
        fbank[m-1, k] = (bins[m+1] - k) / (bins[m+1] - bins[m])

```

Figure 18 construction of triangular filters in python.

The following step was to apply this filter onto the current array of frames in the power spectrum. This was achieved by applying a matrix multiplication between the array of power spectrum frames with the transpose of the filter bank array. The resultant array is row represents a frame with each column representing its energy in one of the 40 Mel bands. A calculation is then done to prevent an error if any zero values are present. These filter bank energies are then converted to the decibel scale giving the final log mel spectrogram ready for input into the CNN.

```

filter_banks = np.dot(pow_frames, fbank.T)
filter_banks = np.where(filter_banks == 0,
                        np.finfo(float).eps, filter_banks)

return 20 * np.log10(filter_banks)

```

Figure 19 final step in computing the log mel spectrogram pf a one second audio file.

3.2 Implementation of Convolutional Neural Network

As mentioned previously in section 2.1.1 this CNN was trained on the Google V2 speech command dataset. This dataset contains several thousand iterations of each word across many different

speakers and noise levels ideal for isolated voice recognition. Using the Keras sequential model using this dataset a Python script was created to train the CNN.

3.2.1 Dataset preparation

The data provided by google in the speech command dataset was stored as .wav files. For the data to be input to a CNN model it must be first converted into a mel spectrogram via the audio processing pipeline previously outlined in section 3.1.1. The data is pulled from the directory and labelled. The batch size is set to 64 which instructs the model to process 64 samples at a time in training. The data is then split into 80% training data and 20% validation data. The validation data is then further split between validation and testing giving a final split of 80% training, 10% validation and 10% testing in figure x. The output length is set to 16,000 samples standardising each audio input to 1 second of 16kHz audio.

```
train_ds, val_ds = tf.keras.utils.audio_dataset_from_directory(  
    directory=data_dir,  
    batch_size=64,  
    validation_split=0.2,  
    seed=0,  
    output_sequence_length=16000,  
    subset='both')
```

Figure 20 data preparation for CNN.

```
test_ds = val_ds.shard(num_shards=2, index=0)  
val_ds = val_ds.shard(num_shards=2, index=1)
```

Figure 21 splitting of test and validation data.

A function was then run to process the audio data through the pipeline outline in section x computing the mel spectrogram of each audio file. The result of this is a set of features of size 99 x 40 which is time frames and frequency bands. An additional channel is then added resulting in a 99 x 40 x 1 tensor mimicking the format of a greyscale image which is perfect for input to a CNN [30]. Normalisation was then performed on the data to prepare the dataset for training. The normalisation function used calculates mean and standard deviation across the entire dataset prior to training ensuring a mean of 0 and standard deviation of 1.

Mean:

$$\mu = \frac{\sum x_i}{N}$$

Standard deviation:

$$\sigma = \sqrt{\frac{\sum (x_i - \mu)^2}{N}}$$

```
def spec_ds(ds):
    def map_fn(audio, label):
        spectrogram = tf.py_function(
            func=retrieve_mel_spectrogram,
            inp=[audio],
            Tout=tf.float32)
        spectrogram.set_shape([99, 40])
        spectrogram = tf.expand_dims(spectrogram, axis=-1)
        return spectrogram, label
    return ds.unbatch().map(
        map_fn, num_parallel_calls=tf.data.AUTOTUNE).batch(64)

train_spectrogram_ds = spec_ds(train_ds)
val_spectrogram_ds = spec_ds(val_ds)
test_spectrogram_ds = spec_ds(test_ds)
```

Figure 22 compute mel spectrogram prior to CNN training.

```
norm_layer = layers.Normalization()
norm_layer.adapt(
    data=train_spectrogram_ds.map(map_func=lambda spec, label: spec))
```

Figure 23 normalisation prior to training.

3.2.2 CNN architecture

Once the data processing had been implemented the architecture could then be devised. The input is a 2D array containing 99 frames by 40 mel bands (99,40,1). This section will describe the implementation and choices made in configuring this architecture.

The implementation used a Keras Sequential model, this was chosen as it is a simple model where data flows from one layer to the next in order as opposed to more complex architectures [24]. A resizing layer was first applied to reduce the input from 99x40 to 32x32, reducing the computational cost of subsequent layers. A normalisation layer was then applied which automatically adapted to the training data, centring the values around zero with unit variance. Two convolutional layers were added, the first having 32 filters and the second 64. The reason for doubling the filter number was to first catch general patterns in the first layer where more complex patterns were discerned in the second. This method has proved to be very effective in training models particularly for CNNs for image classification [31]. The ReLU activation function was used to set any negative values to zero, improving efficiency [32].

$$ReLU(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

```

model = models.Sequential([
    layers.Input(shape=input_shape),
    layers.Resizing( height: 32, width: 32),
    norm_layer,
    layers.Conv2D( filters: 32, kernel_size: 3, activation='relu'),
    layers.Conv2D( filters: 64, kernel_size: 3, activation='relu'),

```

Figure 24 Convolutional layers of the CNN.

Max pooling was then implemented which creates a 2x2 window that slides across each feature map keeping only the maximum value in each window. This halves the output dimensions, reducing the amount of data processed by subsequent layers as well as reducing the effect of small shifts in position on the features. A dropout layer set to 0.25 was applied after pooling, randomly setting 25% of values to zero at each training step to reduce overfitting.

The feature maps were then flattened into a single one-dimensional vector. This was necessary as the subsequent dense layer requires a fixed size 1D input. The flattened vector was then input into a dense layer with 128 neurons using ReLU activation. This layer learns which combinations of features correspond to each word. A second dropout layer set to 0.5 was applied, randomly setting 50% of the neurons to zero at each training step, stopping the network from relying on any one neuron. This was turned off during inference. The final dense layer outputs a raw value for each of the 10 command words without a SoftMax activation, as the loss function handles the conversion to probabilities internally for improved numerical stability.

```

    layers.MaxPooling2D(),
    layers.Dropout(0.25),
    layers.Flatten(),
    layers.Dense( units: 128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(num_labels),
])

```

Figure 25 final layers of the CNN.

The next step performed was the configuration of the training process. The optimiser chosen was the Adam optimiser with the default learning rate of 0.001. This optimiser adjusts its learning rate for each individual weight making it useful for this scenario where gradients can vary widely across weights caused by differences in mel bands and speech characteristics. It also has a proven ability in handling noise [33], which is necessary for training models with small data sizes.

The loss function was then chosen to be sparse categorical cross entropy with the from_logits parameter set to true. This setting means the model outputs raw values rather than probabilities, with the SoftMax conversion being performed internally by the loss function which is more numerically stable. This function accepts integer labels compared to standard categorical cross entropy which requires a vector for each label. Cross entropy was chosen over mean squared error (MSE) as cross entropy produces larger gradients with large prediction errors leading to better performance in classification tasks [34]. An early stopping callback was implemented to end the training process if validation loss did not improve over 2 consecutive epochs. The model was trained for a maximum 10 epochs. The output shape for each layer along with parameters can be seen in the model of this CNN figure 27.

```

EPOCHS = 10
print("\nTraining has begun.")
history = model.fit(
    train_spectrogram_ds,
    validation_data=val_spectrogram_ds,
    epochs=EPOCHS,
    callbacks=[
        tf.keras.callbacks.EarlyStopping(verbose=1, patience=2),
        tensorboard_callback
    ],
)

```

Figure 26 CNN callback function.

Layer (type)	Output Shape	Param #
resizing (Resizing)	(None, 32, 32, 1)	0
normalization (Normalization)	(None, 32, 32, 1)	3
conv2d (Conv2D)	(None, 30, 30, 32)	320
conv2d_1 (Conv2D)	(None, 28, 28, 64)	18,496
max_pooling2d (MaxPooling2D)	(None, 14, 14, 64)	0
dropout (Dropout)	(None, 14, 14, 64)	0
flatten (Flatten)	(None, 12544)	0
dense (Dense)	(None, 128)	1,605,760
dropout_1 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Figure 27 Model of CNN.

3.3 Real time audio processing

3.3.1 Flutter application

A flutter application was then developed which takes an audio input and then passes this data on to a python backend. This record package of this application was used to capture audio from the devices microphone as a continuous stream of PCM16 16kHz single channel audio data. The configuration was chosen as it matches the format of the Google Speech command dataset used to train the CNN.

A WebSocket connection was also implemented to interface between the python server which performs the audio processing. A WebSocket connection was used as it gives a continuous connection between app and server allowing chunks of audio data to be sent immediately to the server. Once the audio is processed on the server a character is sent back to the application. This character is then displayed in the application and forwarded via BLE to the esp32 controlling the robot. Figure 28 shows the UI of this application with a button to hold down which begins the streaming of audio to the python server.

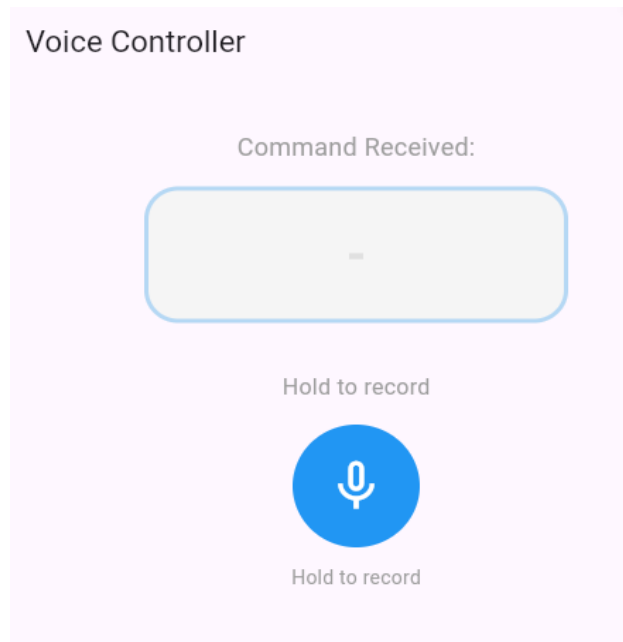


Figure 28 UI of mobile application.

3.3.2 Python webserver

The Python webserver first established a connection with the flutter app via WebSocket and awaits audio chunks to be streamed to it. The audio chunks are then held in a buffer until the buffer size is equivalent of 1 second long in bytes. The buffer is checked to ensure there is valid data within it and is processed through the pipeline to produce its mel spectrogram previously illustrated in section x. Once the audio is converted to mel spectrogram it is then ran through the trained CNN illustrated in section x. The output is given a percentage score for what word it is likely to be. If the word is predicted with a likelihood above 75% then a char is transmitted back to the application representing the predicted word.

3.4 Hardware implementation

Initially the motors, motor driver and distance sensor were tested as is covered in section 4 of the report. Once initial set up and testing of the electronic components was completed the robot unit could be assembled. The first step of assembly was to take the piece of MDF board and to connect the n20 motors the bottom using the mounts included with the motors. The metal ball caster wheels were then attached to the bottom of board with spacing blocks glued between the board and wheels to ensure adequate clearance shown in figure 29. Two rubber wheels were attached to the n20 motors.



Figure 29 MDF board with motors, wheels and ball casters attached.

The electronic components were then spread over two breadboards one small breadboard at the front to mount the ultrasonic distance sensor. A large breadboard was placed in the centre to hold the motor driver, esp32 and lipo rider plus. All components were then wired in accordance with figures 30-32.

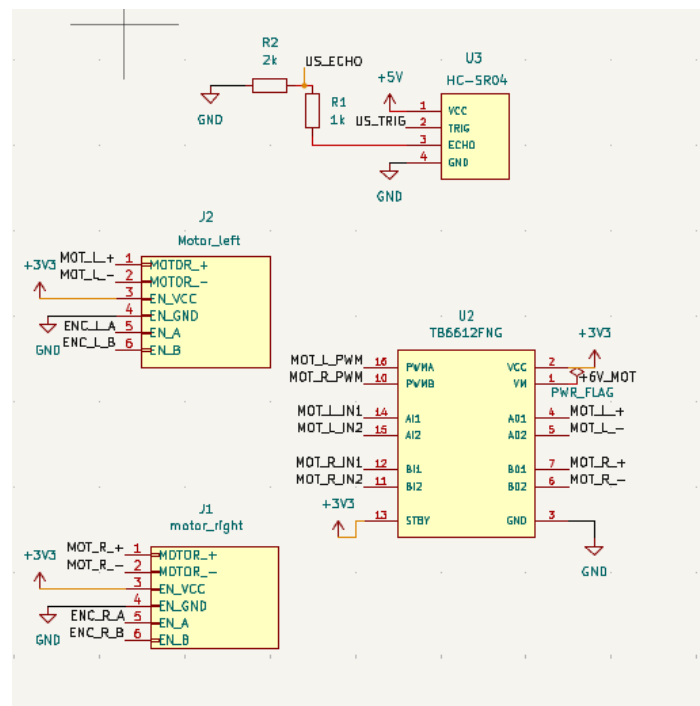


Figure 30 schematic view of TB6612FNG (U2), n20 motor left (J2), n20 motor right (J1) and ultrasonic distance sensor (U3).

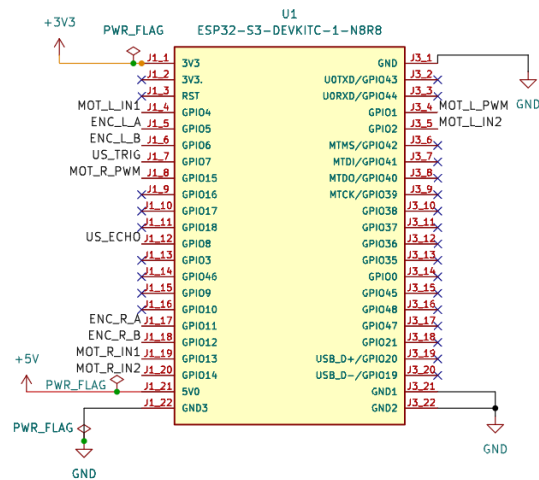


Figure 31 schematic view of esp32 (U1).

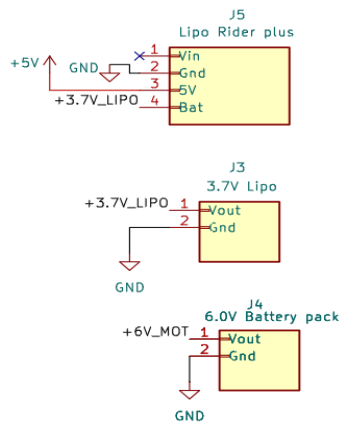


Figure 32 schematic view of lipo rider plus (J5), 3.7V lipo battery (J3) and 6V battery pack (J4).

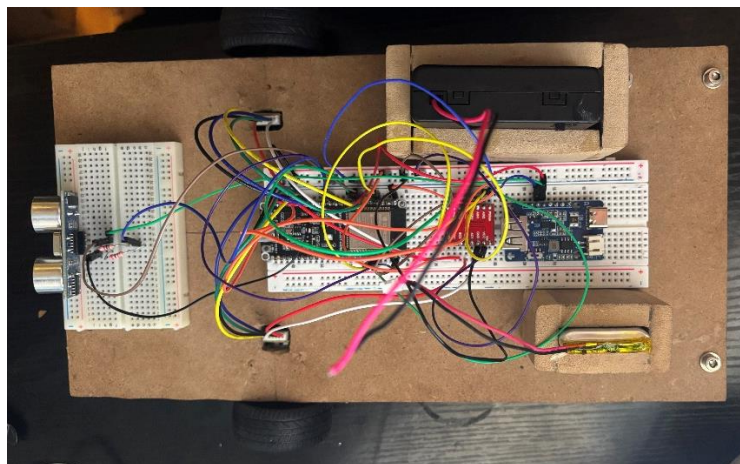


Figure 33 Robotic unit.

3.5 Implementing robot movement

Once the robotic units had been assembled and initial motor and sensor testing was complete an Arduino script was created to create a Bluetooth connection with a mobile phone to send commands which were then translated into movements of the robot. This was achieved by using the Nordic

UART service [35]. The esp32 acts as a Bluetooth receiver advertising itself through the RX characteristic UUID shown in figure 20.

```
#define SERVICE_UUID          "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"  
#define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
```

Figure 34 service UUID and RX characteristic UUID for Nordic UART.

A callback function seen in figure 20 was created to take the character transmitted via Bluetooth and translate it to a command. This class is created which inherits from the BLE library BLECharacteristicCallbacks class. This class takes the character transmitted via the mobile app. Based off the received char it either the speed is changed with character 1-5 or directions are assigned F for forwards B for backwards etc.

```
class MyCallbacks: public BLECharacteristicCallbacks {  
    void onWrite(BLECharacteristic *pCharacteristic) {  
        String rxValue = pCharacteristic->getValue();  
  
        if (rxValue.length() > 0) {  
            char receivedChar = rxValue[0]; // Look at the first character sent  
            Serial.print("Received BLE Command: ");  
            Serial.println(receivedChar);  
  
            // control speed based off recieved number  
            if (receivedChar == '1') currentSpeed = 100;  
            else if (receivedChar == '2') currentSpeed = 140;  
            else if (receivedChar == '3') currentSpeed = 180;  
            else if (receivedChar == '4') currentSpeed = 220;  
            else if (receivedChar == '5') currentSpeed = 255;  
  
            // Directional Commands  
            else if (receivedChar == 'F' || receivedChar == 'B' ||  
                    receivedChar == 'L' || receivedChar == 'R' ||  
                    receivedChar == 'S') {  
                currentCommand = receivedChar;  
            }  
        }  
    }  
}
```

Figure 35 MyCallbacks class Arduino.

The BLE connection must then be setup. The first step was initialising the device shown in figure 21 the name shown to other devices to connect is ESP32_ROBOT. The next step was to create a server instance this allows other devices to now interact with the esp32. A Nordic UART service was then created allowing any device to communicate via Nordic UART standard. A characteristic is then created using the RX UUID allowing a device to write data via BLE. Each time the device is written to the onWrite function is then automatically triggered. The server is then started and it begins broadcasting to nearby devices.

```

// BLE Setup
Serial.println("Starting BLE Server...");
BLEDevice::init("ESP32_ROBOT");
BLEServer *pServer = BLEDevice::createServer();

BLEService *pService = pServer->createService(SERVICE_UUID);

BLECharacteristic *pRxCharacteristic = pService->createCharacteristic(
    CHARACTERISTIC_UUID_RX,
    BLECharacteristic::PROPERTY_WRITE
);

pRxCharacteristic->setCallbacks(new MyCallbacks());
pService->start();

// Start broadcasting
pServer->getAdvertising()->start();
Serial.println("BLE Ready. Waiting for connection. ");

```

Figure 36 BLE setup in Arduino.

The rest of the code then translates the directional commands to motor outputs e.g left motor is high while right motor is low to turn left. The ultrasonic sensor also checks for objects every 20ms. Testing of this code is illustrated in section 4.

4.0 Testing

4.1 ESP-32 with motor driver and motor

4.1.1 Motor test 1

Initially a single N20 motor was wired to both esp32 and TB6612FNG motor driver in the configuration shown in figure x. An Arduino script was then run on the esp32. The intention of this script was to simply rotate the motor forwards for 2 seconds then stop and then backwards for 2 seconds. The script then reads the encoder count as the motors rotate and outputs this to the serial monitor. The intent of this test is to ensure the readings are staying consistent over several iterations.

```

23:29:15.362 -> Ready to begin testing.
23:29:15.362 -> Motor has moved 0 Times.
23:29:15.362 -> Motor forward. Encoders: -4120
23:29:16.355 -> Moving Backward... Encoders: -346
23:29:19.345 -> Motor has moved 1 Times.
23:29:19.345 -> Motor forward. Encoders: -4079
23:29:22.331 -> Moving Backward... Encoders: -302
23:29:25.357 -> Motor has moved 2 Times.
23:29:25.357 -> Motor forward. Encoders: -4036
23:29:28.339 -> Moving Backward... Encoders: -263
23:29:31.325 -> Motor has moved 3 Times.
23:29:31.325 -> Motor forward. Encoders: -3992
23:29:34.359 -> Moving Backward... Encoders: -222
23:29:37.335 -> Motor has moved 4 Times.
23:29:37.335 -> Motor forward. Encoders: -3956
23:29:40.359 -> Moving Backward... Encoders: -176
23:29:43.337 -> Motor has moved 5 Times.
23:29:43.337 -> Motor forward. Encoders: -3912
23:29:46.318 -> Moving Backward... Encoders: -139
23:29:49.342 -> Motor has moved 6 Times.
23:29:49.342 -> Motor forward. Encoders: -3871
23:29:52.329 -> Moving Backward... Encoders: -97

```

Figure 37 Output in serial monitor of motor rotation test.

The output from the motor test shows the encoder values drifting forward over each iteration. In the forward direction, counts decreased from -4120 to -3785 across 10 cycles, while the backward counts decreased from -346 to -12.

4.1.2 Motor test 2

The built-in pull up resistors in the esp32 [15] were implemented and interrupt function was changed to use the esp32's hardware pulse counter via the ESP32Encoder library [36]. The encoder values produced for forward rotation were negative to address this issue the c1 and c2 wires were swapped as the phase between these two channels determines the counting direction.

```
Motor forward. Encoders: Ready to begin testing.
Motor has moved 0 Times.
Motor forward. Encoders: 4171
Moving Backward... Encoders: 423
Motor has moved 1 Times.
Motor forward. Encoders: 4202
Moving Backward... Encoders: 458
Motor has moved 2 Times.
Motor forward. Encoders: 4240
Moving Backward... Encoders: 491
Motor has moved 3 Times.
Motor forward. Encoders: 4273
Moving Backward... Encoders: 524
Motor has moved 4 Times.
Motor forward. Encoders: 4308
Moving Backward... Encoders: 556
Motor has moved 5 Times.
Motor forward. Encoders: 4340
Moving Backward... Encoders: 587
```

Figure 38 Output from motor test 2.

The output shows the encoder count was now counting positive for forward rotation and negative for backward rotation. The issue of drifting encoder values persists in these results following the addition of the pull up resistors and hardware pulse counter. The persistence of the problem indicates this issue is unlikely an issue caused by electrical noise and more likely attributed to slight differences between the speed of the motors in forward and backward rotation.

4.1.3 Motor test 3

The code now used the pulses instead of time to stop set the backwards and forward limits.

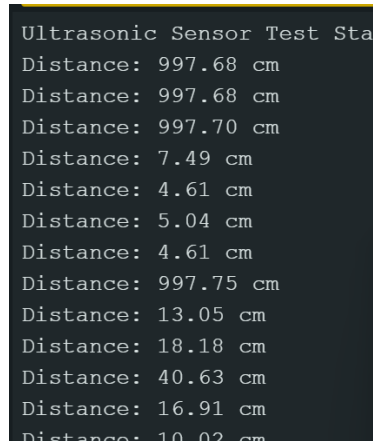
```
Ready to begin testing.
Motor has moved 0 Times.
Motor forward. Encoders: 4000
Moving Backward... Encoders: 0
Motor has moved 1 Times.
Motor forward. Encoders: 4000
Moving Backward... Encoders: 0
Motor has moved 2 Times.
Motor forward. Encoders: 4000
Moving Backward... Encoders: 0
Motor has moved 3 Times.
Motor forward. Encoders: 4000
Moving Backward... Encoders: 0
Motor has moved 4 Times.
Motor forward. Encoders: 4000
Moving Backward... Encoders: 0
```

Figure 39 Output for motor test 3.

The results of motor test 3 now shows the motor is rotating forward for exactly 4000 pulses and then rotating backward until the total pulses reaches 0. These results are in line with the expected outcome.

4.2 ESP-32 with HC-SRO4

The objective of this test is to connect the HC-SR04 to the esp32 and get reliable distance readings from the sensor. This test was implemented by connecting the HC-SR04 to the esp32 as shown in figure 40. A simple Arduino script was implemented to output the distance measured by the sensor in the Arduino serial monitor.



```
Ultrasonic Sensor Test Sta
Distance: 997.68 cm
Distance: 997.68 cm
Distance: 997.70 cm
Distance: 7.49 cm
Distance: 4.61 cm
Distance: 5.04 cm
Distance: 4.61 cm
Distance: 997.75 cm
Distance: 13.05 cm
Distance: 18.18 cm
Distance: 40.63 cm
Distance: 16.91 cm
Distance: 10.02 cm
```

Figure 40 Output from HC-SR04 distance test.

The test successfully showed output distance in the Arduino serial monitor. As objects were placed close to the sensor they were registered in the output.

4.3 Stopping test

This test was an important milestone in ensuring the robot could avoid objects if placed in its path. The objective was to set a stopping distance for any object detected within 15cm of the sensor. Initially this was implemented by outputting a warning message in the Arduino serial monitor if any object was detected within the distance stated. This function was added to motor testing script so that the motor would run backwards and forwards unless an object was detected.

```

Motor forward. Encoders: Ready to begin testing.
Motor has moved 0 Times.
Motor forward. Encoders:
OBSTACLE DETECTED at 5.93 cm! Stopping early.
1570
Moving Backward... Encoders: 0
Motor has moved 1 Times.
Motor forward. Encoders:
OBSTACLE DETECTED at 9.21 cm! Stopping early.
553
Moving Backward... Encoders: 0
Motor has moved 2 Times.
Motor forward. Encoders:
OBSTACLE DETECTED at 5.20 cm! Stopping early.
2080
Moving Backward... Encoders: 0
Motor has moved 3 Times.
Motor forward. Encoders:
OBSTACLE DETECTED at 9.54 cm! Stopping early.
-44
Moving Backward... Encoders: -44
Motor has moved 4 Times.
Motor forward. Encoders:
OBSTACLE DETECTED at 8.45 cm! Stopping early.
2242
Moving Backward... Encoders: 0
Motor has moved 5 Times.
Motor forward. Encoders:
OBSTACLE DETECTED at 8.20 cm! Stopping early.
97

```

Figure 41 Output from motor stopping test.

The stopping function was successfully implemented as seen in figure 41 as the motors ran as normal until an object was detected in which case the motor paused until the obstruction was cleared.

4.4 Bluetooth connectivity

A mobile application was downloaded to initially test the BLE connection and send commands to the esp32. The name of the mobile application is nRF connect and it is created by Nordic Semiconductor [35]. This application is used for the testing and development of Bluetooth devices. This application was used to connect to the esp32 BLE advertised as ESP32_ROBOT.

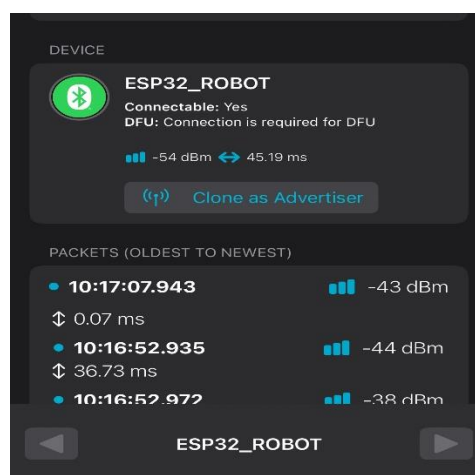


Figure 42 ESP32 advertising its BLE connection.

```

Starting BLE Server...
BLE Ready. Waiting for connection.
Received BLE Command: F
OBSTACLE AT 11.42 cm! Emergency Stop.
ReceiveF
Received BLE Command: L
B
Received BLE Command: R
Received BLE Command: S

```

Figure 43 Output from BLE testing.

4.5 CNN testing

The classification report, confusion matrix and graph of accuracy vs loss of the CNN were created after training had completed. It can be seen in figure 30 that the CNN quickly gained accuracy in prediction in the first few epochs with it tapering off exponentially as training proceeded. Loss also dropped drastically in the first epochs for both validation and testing data. The confusion matrix figure 31 shows the performance of the CNN for each word against the testing data. The word forward was mistaken for the word four 37 times and four for forward 16 times showing an issue in classification of this word. The other words performed with a high accuracy in relation to test data.

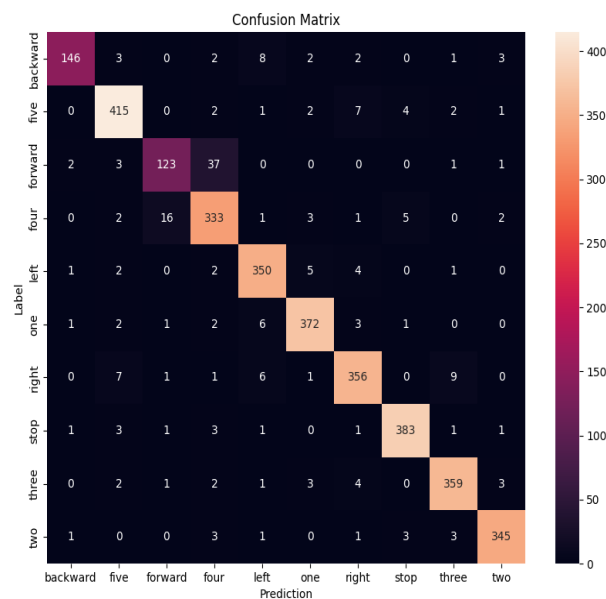


Figure 44 Confusion matrix output at the end of CNN training.

The code was then tested using the authors own voice recording trimmed and prepared in similar fashion to the google dataset. 20 samples of each word were used to test this data a test was then ran with clean data and data with noise added to simulate a real-life environment. The confusion matrix in figure 32 shows the result of testing these words against the CNN. A function was then added to add noise to mimic a moderately noisy indoor environment seen in figure 33. From the confusion matrix it can be seen that the noiseless example right and stop got an accuracy of 94.5% an accuracy of with five achieving an accuracy of 88.25% all other words achieved 100% accuracy.

When noise was introduced five then increased in accuracy to 94.5% whereas right dropped to 83.33% accuracy.

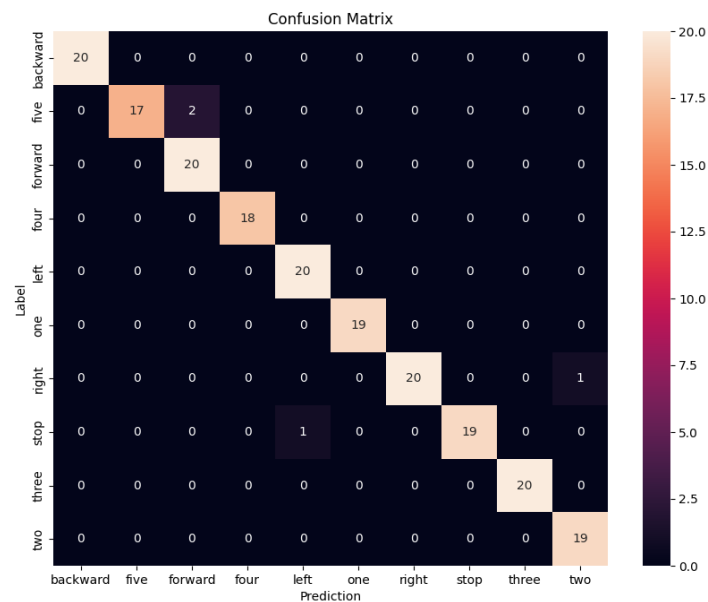


Figure 45 Confusion matrix tested using data outside of Google speech dataset.

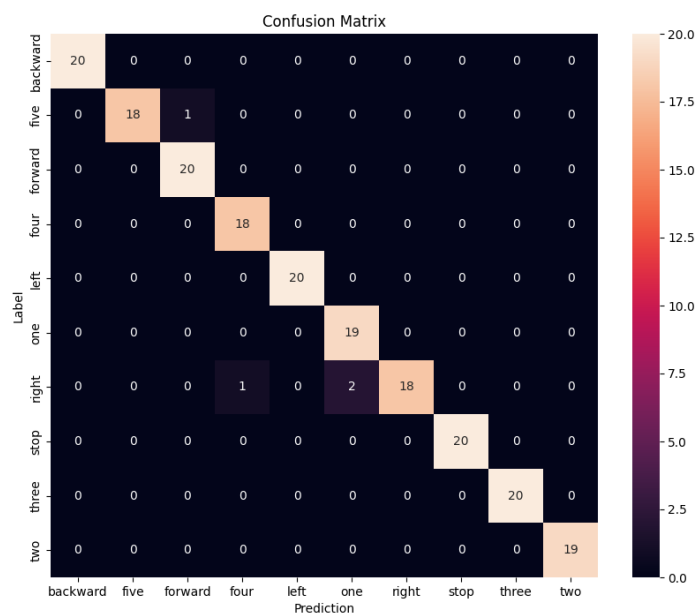


Figure 46 Noisy word prediction confusion matrix.

4.6 Integration testing

4.6.1 System test

The whole system was tested to ensure all parts worked together in unison. This was performed in by running flutter as a chrome app which then connected to the real time audio script via a WebSocket. The pipeline that has been described in section 2 and 3 was then employed to give a prediction on the word spoken. This char representing the spoken word was then transmitted via BLE to the esp32

which then controlled the robot's movements. A screen recording of the app and recording of robot movement have been submitted to the project demonstration submission. The system was successful in responding to all voice commands programmed.

5.0 Conclusion

5.1 Evaluation

The overall objectives set out in the objective of this project were met with a robot assembled which can respond to all voice commands outlined in the introduction. A demonstration of the operating system has been recorded and submitted demonstrating these objectives have been achieved.

The robot responds to voice commands with a relatively low latency, the voice recognition software has been shown to be accurate in predicting the word spoken and the overall system performs well as a whole. The steps taken across this project can be accessed in the logbook attached.

5.2 Future improvements

5.2.1 Edge computing

A possible upgrade to be added to this project would be to move the processing of audio over to the microcontroller on the robot unit. This would make this project applicable across many more fields and applications as the need for a Wi-Fi connection would be eliminated. This however would require a large amount of memory and computation to be performed on the microcontroller.

List of acronyms

STT: Speech to Text

ASR: Automatic Speech Recognition

DTW: Dynamic Time Warping

WER: Word Error Rate

CNN: Convolutional Neural Networks

HMM: Hidden Markov Models

IOT: Internet of Things

ML: Machine Learning

AI: Artificial Intelligence

NN: Neural Networks

BLE: Bluetooth Low Energy

GPIO: General Purpose Input/Output

BJT: Bipolar Junction Transistor

MOSFET: Metal-Oxide-Semiconductor Field-Effect Transistor

PID: Proportional-Integral-Derivative

PCM: Pulse Code Modulated

FFT: Fast Fourier Transform

ReLU: Rectified Linear Unit

MSE: Mean Squared Error

UI: User Interface

MDF: Medium-Density Fibreboard

UUID: Universally Unique Identifier

References

[1]

S. J. Arora and H. Rishi Pal Singh, "Automatic Speech Recognition: A Review," 2012.

[2]

"Sampling and Quantization."

[3]

C. S. Myers, "A COMPARATIVE STUDY OF SEVERAL DYNAMIC TIME WARPING ALGORITHMS FOR SPEECH RECOGNITION," 1980.

[4]

A. Radford, J. W. Kim, T. Xu, G. Brockman, C. Mcleavey, and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision", Accessed: Mar. 31, 2026. [Online]. Available: <https://github.com/openai/>

[5]

A. L. Samuel, "Some studies in machine learning using the game of checkers," *IBM J Res Dev*, vol. 3, no. 3, pp. 210–229, Jul. 1959, doi: 10.1147/RD.33.0210.

[6]

D. Pandey, K. Niwaria, and B. Chourasia, "Machine Learning Algorithms: A Review," *International Research Journal of Engineering and Technology*, 2008, Accessed: Apr. 11, 2026. [Online]. Available: www.irjet.net

[7]

Y. Li, "Deep Reinforcement Learning: An Overview," Jan. 2017, Accessed: Apr. 11, 2026. [Online]. Available: <https://arxiv.org/pdf/1701.07274>

[8]

J. G. Cabello, "Mathematical Neural Networks," *Axioms*, vol. 11, no. 2, Feb. 2022, doi: 10.3390/axioms11020080.

[9]

"Deep Learning." Accessed: Apr. 11, 2026. [Online]. Available: <https://www.deeplearningbook.org/>

[10]

A. Krenker, J. Bešter, A. K.-A. N. Networks, and undefined 2011, "Introduction to the artificial neural networks," *researchgate.net*, Accessed: Apr. 11, 2026. [Online]. Available: https://www.researchgate.net/profile/Kenji-Suzuki-2/publication/319316102_Artificial_Neural_Networks_-_Methodological_Advances_and_Biomedical_Applications/links/59a42f16aca272a6461bb35e/Artificial-Neural-Networks-Methodological-Advances-and-Biomedical-Applications.pdf#page=15

[11]

W. Samek, G. Montavon, S. Lapuschkin, C. J. Anders, and K. R. Müller, "Explaining Deep Neural Networks and Beyond: A Review of Methods and Applications," *Proceedings of the IEEE*, vol. 109, no. 3, pp. 247–278, Mar. 2021, doi: 10.1109/JPROC.2021.3060483.

[12]

"View of Artificial Neural Networks: An Overview." Accessed: Apr. 12, 2026. [Online]. Available: <https://mesopotamian.press/journals/index.php/cs/article/view/118/121>

[13]

K. Janocha and W. M. Czarnecki, "On Loss Functions for Deep Neural Networks in Classification," *Schedae Informaticae*, vol. 25, pp. 49–59, Feb. 2017, doi: 10.4467/20838476SI.16.004.6185.

[14]

T. Dietterich, "Overfitting and undercomputing in machine learning Citation in BibTeX format," *ACM Computing Surveys (CSUR)*, vol. 27, no. 3, 1995, doi: 10.1145/212094.212114.

[15]

"ESP32-S3 Datasheet | Espressif Documentation." Accessed: Feb. 08, 2026. [Online]. Available: [https://documentation.espressif.com/esp32-s3_datasheet_en.html#\[59,%22XYZ%22,56.69,692.84,null](https://documentation.espressif.com/esp32-s3_datasheet_en.html#[59,%22XYZ%22,56.69,692.84,null)

[16]

"UNO R3 | Arduino Documentation." Accessed: Mar. 01, 2026. [Online]. Available: <https://docs.arduino.cc/hardware/uno-rev3/#tech-specs>

[17]

P. By ALLDATASHEETCOM, "Toshiba Bi-CD Integrated Circuit Silicon Monolithic TB6612FNG Driver IC for Dual DC motor," 2014.

[18]

"L298N pdf, L298N Description, L298N Datasheet, L298N view ::: ALLDATASHEET ::: " Accessed: Mar. 01, 2026. [Online]. Available: <https://www.alldatasheet.com/datasheet-pdf/view/22440/STMICROELECTRONICS/L298N.html>

[19]

"DCGM-N20-12V-EN-200RPM - Waveshare Wiki." Accessed: Mar. 02, 2026. [Online]. Available: <https://www.waveshare.com/wiki/DCGM-N20-12V-EN-200RPM>

[20]

"Ultrasonic Ranging Module HC-SR04." [Online]. Available: www.Electfreaks.com

[21]

"Lipo Rider Plus (Charger/Booster) - 5V/2.4A USB Type C - Seeed Studio." Accessed: Apr. 10, 2026. [Online]. Available: https://www.seeedstudio.com/Lipo-Rider-Plus-p-4204.html?srltid=AfmBOora1_1bPBlieMVMqkSAw5KQ2F0Ye1ym23nM3ijtUyvXk_DUIIzO

[22]

"VOSK Offline Speech Recognition API." Accessed: Apr. 11, 2026. [Online]. Available: <https://alphacephei.com/vosk/>

[23]

C. K. Wei and H. Nugroho, "Design of a DNN-based operator on edge device for keyword spotting," *Advances in Computational Intelligence*, vol. 5, no. 2, Jun. 2025, doi: 10.1007/s43674-025-00080-2.

[24]

"The Sequential model." Accessed: Apr. 07, 2026. [Online]. Available: https://keras.io/guides/sequential_model/

[25]

"Flutter - Build apps for any screen." Accessed: Apr. 13, 2026. [Online]. Available: <https://flutter.dev/>

[26]

"Mel-frequency Cepstral Coefficients (MFCC) for Speech Recognition - GeeksforGeeks." Accessed: Apr. 06, 2026. [Online]. Available: <https://www.geeksforgeeks.org/nlp/mel-frequency-cepstral-coefficients-mfcc-for-speech-recognition/>

[27]

D. Anggraeni, W. S. M. Sanjaya, M. Y. S. Nurasyidiek, and M. Munawwaroh, "The Implementation of Speech Recognition using Mel-Frequency Cepstrum Coefficients (MFCC) and Support Vector Machine (SVM) method based on Python to Control Robot Arm," in *IOP Conference Series: Materials Science and Engineering*, Institute of Physics Publishing, Jan. 2018. doi: 10.1088/1757-899X/288/1/012042.

[28]

D. M. Reddy and S. Reddy, "Effects of padding on LSTMs and CNNs," Mar. 2019, Accessed: Apr. 06, 2026. [Online]. Available: <https://arxiv.org/pdf/1903.07288>

[29]

“How to Calculate MFCCs.” Accessed: Apr. 06, 2026. [Online]. Available: <https://apxml.com/courses/applied-speech-recognition/chapter-2-feature-extraction-for-speech/calculating-mfccs>

[30]

W. Rawat and Z. Wang, “Deep convolutional neural networks for image classification: A comprehensive review,” *Neural Comput*, vol. 29, no. 9, pp. 2352–2449, Sep. 2017, doi: 10.1162/NECO_A_00990.

[31]

K. Simonyan and A. Zisserman, “VERY DEEP CONVOLUTIONAL NETWORKS FOR LARGE-SCALE IMAGE RECOGNITION,” 2015, Accessed: Apr. 07, 2026. [Online]. Available: <http://www.robots.ox.ac.uk/>

[32]

Y. Bai, “RELU-Function and Derived Function Review”, doi: 10.1051/shsconf/202214402006

[33]

D. P. Kingma and J. Lei Ba, “ADAM: A METHOD FOR STOCHASTIC OPTIMIZATION”.

[34]

S. Mannor, B. Peleg, and R. Rubinstein, “The cross entropy method for classification,” *ICML 2005 - Proceedings of the 22nd International Conference on Machine Learning*, pp. 561–568, 2005, doi: 10.1145/1102351.1102422.

[35]

“Nordic UART Service (NUS).” Accessed: Apr. 13, 2026. [Online]. Available: <https://docs.nordicsemi.com/bundle/ncs-latest/page/nrf/libraries/bluetooth/services/nus.html>

[36]

“GitHub - madhephaestus/ESP32Encoder: A Quadrature and half quadrature PCNT peripheral driven encoder library supporting 8 encoders · GitHub.” Accessed: Apr. 13, 2026. [Online]. Available: <https://github.com/madhephaestus/ESP32Encoder>

Appendices

Appendix 1: Source code

This full source code for this project has been provided in a zip file alongside this report. The contents are organised as follows:

- CNN - CNN training script (Python)
- Audio processing – Real time audio processing script (Python)
- Flutter application – Mobile application code
- Arduino – ESP32 motor control and BLE

Appendix 2: Log book

The log book kept from the beginning of this project was submitted alongside this report. It includes a table for each week discussing the progress made on a daily basis.